```
// WATERPOINT WATCH — PSEUDOCODE
CONSTANT PRIORITY_RANK = {
    Critical: 1,
    Urgent: 2,
    High: 3,
    Medium: 4,
    Low: 5
}
CONSTANT VALID_STATUSES = [
    New, Verified, Assigned, In_Progress, Resolved, Closed
]
FUNCTION validateReport(report, registeredWaterPoints):
    IF report.water_point_id IS EMPTY:
        RETURN ERROR("Water-point ID is required")

    IF report.water_point_id NOT IN registeredWaterPoints:
        RETURN ERROR("Water-point ID is not registered")

    IF report.issue_type IS EMPTY:
        RETURN ERROR("Issue type is required")

    IF report.severity IS NOT AN INTEGER OR report.severity < 1
       OR report.severity > 5:
        RETURN ERROR("Severity must be an integer from 1 to 5")

    IF report.people_affected IS NOT AN INTEGER
       OR report.people_affected < 1:
        RETURN ERROR("People affected must be a positive integer")

    IF report.days_open IS NOT AN INTEGER OR report.days_open < 0:
        RETURN ERROR("Days open must be a non-negative integer")

    IF report.vulnerable_users IS NOT A BOOLEAN:
        RETURN ERROR("Vulnerable-users value must be true or false")

    RETURN VALID
FUNCTION calculateScore(severity, peopleAffected, daysOpen,
                        vulnerableUsers):
    userPoints = MIN(8, CEILING(peopleAffected / 25))
    waitPoints = MIN(6, daysOpen)

    IF vulnerableUsers = TRUE:
        riskPoints = 5
    ELSE:
        riskPoints = 0

    RETURN (4 * severity) + userPoints + waitPoints + riskPoints
```

```
FUNCTION classifyReport(report):
    report.score = calculateScore(
        report.severity,
        report.people_affected,
        report.days_open,
        report.vulnerable_users
    )
    report.verification_flag = FALSE

    IF report.severity = 5
       OR report.issue_type = suspected_contamination:
        report.priority = Critical
        report.verification_flag = TRUE
    ELSE IF report.score >= 27:
        report.priority = Urgent
    ELSE IF report.score >= 18:
        report.priority = High
    ELSE IF report.score >= 9:
        report.priority = Medium
    ELSE:
        report.priority = Low

    RETURN report
FUNCTION findDuplicate(report, reports):
    FOR EACH existingReport IN reports:
        IF existingReport.status NOT IN [Resolved, Closed]
           AND existingReport.water_point_id = report.water_point_id
           AND existingReport.issue_type = report.issue_type:
            RETURN existingReport

    RETURN NONE
```

```
FUNCTION submitReport(report, reports, registeredWaterPoints,
                      currentDateTime):
    validationResult = validateReport(report, registeredWaterPoints)
    IF validationResult IS ERROR:
        RETURN validationResult

    existingReport = findDuplicate(report, reports)

    IF existingReport IS NOT NONE:
        APPEND report.observation TO existingReport.observations
        existingReport.severity = report.severity
        existingReport.people_affected = report.people_affected
        existingReport.vulnerable_users = report.vulnerable_users
        existingReport.days_open = MAX(
            existingReport.days_open,
            report.days_open
        )
        existingReport.updated_at = currentDateTime
        targetReport = existingReport
    ELSE:
        report.report_id = GENERATE_UNIQUE_ID()
        report.created_at = currentDateTime
        report.updated_at = currentDateTime
        report.status = New
        report.observations = [report.observation]
        APPEND report TO reports
        APPEND_STATUS_HISTORY(report.report_id, New, currentDateTime)
        targetReport = report

    targetReport = classifyReport(targetReport)
    SAVE targetReport

    queue = sortOpenQueue(reports)
    RETURN {
        report: targetReport,
        queue: queue,
        confirmation: [
            targetReport.report_id,
            targetReport.priority,
            targetReport.score,
            targetReport.verification_flag
        ]
    }
FUNCTION sortOpenQueue(reports):
    openReports = EMPTY LIST

    FOR EACH report IN reports:
        IF report.status NOT IN [Resolved, Closed]:
            APPEND report TO openReports

    SORT openReports BY:
        PRIORITY_RANK[report.priority] ASCENDING,
        report.score DESCENDING,
        report.days_open DESCENDING,
        report.report_id ASCENDING

    RETURN openReports
```

```
FUNCTION displayOpenQueue(reports):
    queue = sortOpenQueue(reports)

    FOR EACH report IN queue:
        DISPLAY report.report_id,
                report.water_point_id,
                report.priority,
                report.score,
                report.status,
                report.days_open,
                report.verification_flag
FUNCTION updateStatus(reportId, newStatus, userRole, reports,
                      currentDateTime, repairNote):
    IF userRole IS NOT AUTHORIZED:
        RETURN ERROR("User is not authorized to update reports")

    report = FIND report IN reports WHERE report.report_id = reportId
    IF report IS NONE:
        RETURN ERROR("Report not found")

    allowedNextStatus = {
        New: Verified,
        Verified: Assigned,
        Assigned: In_Progress,
        In_Progress: Resolved,
        Resolved: Closed
    }

    IF newStatus != allowedNextStatus[report.status]:
        RETURN ERROR("Status transition is not allowed")

    report.status = newStatus
    report.updated_at = currentDateTime

    IF repairNote IS NOT EMPTY:
        APPEND repairNote TO report.non_sensitive_notes

    APPEND_STATUS_HISTORY(
        report.report_id,
        newStatus,
        currentDateTime,
        userRole
    )
    SAVE report
    RETURN report
```

```
FUNCTION syncPendingRecords(localRecords, remoteRecords):
    conflicts = EMPTY LIST

    FOR EACH localRecord IN localRecords WHERE sync_status = Pending:
        remoteRecord = FIND record IN remoteRecords
                       WHERE record.report_id = localRecord.report_id

        IF remoteRecord IS NONE:
            UPLOAD localRecord TO remoteRecords
            SET localRecord.sync_status = Synchronized
        ELSE IF localRecord AND remoteRecord HAVE CONFLICTING CHANGES:
            MARK localRecord.sync_status = Conflict_Review
            APPEND [localRecord, remoteRecord] TO conflicts
        ELSE:
            SYNCHRONIZE compatible changes
            SET localRecord.sync_status = Synchronized

    SAVE localRecords
    SAVE remoteRecords
    RETURN conflicts
FUNCTION summarizeReports(reports):
    summary = {
        open: 0,
        assigned: 0,
        in_progress: 0,
        resolved: 0
    }

    FOR EACH report IN reports:
        IF report.status NOT IN [Resolved, Closed]:
            summary.open = summary.open + 1

        IF report.status = Assigned:
            summary.assigned = summary.assigned + 1
        ELSE IF report.status = In_Progress:
            summary.in_progress = summary.in_progress + 1
        ELSE IF report.status = Resolved:
            summary.resolved = summary.resolved + 1

    RETURN summary
```

```
PROCEDURE main():
    reports = LOAD_SAVED_REPORTS()
    registeredWaterPoints = LOAD_REGISTERED_WATER_POINTS()

    WHILE application_is_running:
        IF user_submits_report:
            result = submitReport(
                submitted_report,
                reports,
                registeredWaterPoints,
                CURRENT_DATE_TIME()
            )
            DISPLAY result

        IF authorized_user_requests_queue:
            displayOpenQueue(reports)

        IF authorized_user_updates_status:
            result = updateStatus(
                report_id,
                new_status,
                user_role,
                reports,
                CURRENT_DATE_TIME(),
                repair_note
            )
            DISPLAY result

        IF authorized_user_requests_summary:
            DISPLAY summarizeReports(reports)

        IF network_connection_available:
            conflicts = syncPendingRecords(localRecords, remoteRecords)
            DISPLAY conflicts FOR authorized_review
```